

ASSIGNMENT 3

AI Capsule

Cloud-Deployed AI Prompt Manager

Full-Stack CRUD + OAuth + JWT + Real Cloud Deployment · React + Node/Express

Designed by Dr Shuo Ding

Subject Coordinator · CSE3CWA / CSE5006

Semester 2, 2026

1. Overview

In this assignment, you will design, build and deploy AI Capsule, a small full-stack web application for saving and managing useful AI prompts. The application is deliberately compact so that you can concentrate on the complete full-stack workflow rather than on building a large product.

A signed-in user can create a prompt record, view their saved records, update an existing record and delete a record. Each record stores the prompt itself together with simple information such as its project, version, category, usefulness, review status, notes and optional screenshot evidence.

The completed application must run on a **real public cloud platform**. Users sign in through OAuth, the Express backend issues and verifies the application JWT, and the backend protects the capsule API so that only an authenticated user can access that user's own records.

2. What You Are Building

AI Capsule is a private prompt library. Students commonly use tools such as ChatGPT, Copilot, Gemini and Claude for coding, writing, debugging and study, but useful prompts are easily lost. AI Capsule gives each authenticated user one place to keep, review and improve their own prompt records.

The required application is intentionally small. It needs a public landing page, OAuth login, a protected dashboard and complete CRUD for prompt records. CREATE adds a new record; READ lists the authenticated user's records; UPDATE changes an existing record; DELETE removes a record. Search, filtering, charts, file upload and other extra features are optional.

3. Assignment Focus

This is a **deployment-first** full-stack assignment. You are assessed on whether the React frontend, Express backend, database, OAuth login, JWT protection and CRUD behaviour work together on the public Internet. Marks are based on the requirements specified in this assignment. Using a more difficult cloud platform, a more complex database solution, or adding features beyond the stated requirements will not result in higher marks.

A localhost-only submission may receive marks for the Full-Stack CRUD section; however, localhost execution does not provide evidence of the assessed cloud deployment, deployed authentication, protected cloud API, or cloud configuration requirements. Therefore, a submission without verifiable deployment to a real public cloud platform is **capped at a maximum mark of 35/100**.

4. Technology (required)

Component	Requirement
Frontend	React
Backend	Node.js + Express

Component	Requirement
Authentication	GitHub OAuth strongly recommended; Google OAuth accepted as fallback
Application session	JWT issued by Express after OAuth; stored in a Secure, HttpOnly cookie named token
Storage	SQLite minimum; PostgreSQL or another relational cloud database accepted
Deployment	Real public cloud / hosting platform; localhost is not cloud deployment evidence
Evidence	Source-code ZIP + README + recorded video + required cURL results

Firebase Authentication is NOT permitted because the Express-side OAuth-to-JWT flow is an assessed part of this assignment. A custom username/password registration system is NOT required. The required JWT is the application JWT issued by your Express backend after successful OAuth authentication; it is not the GitHub or Google OAuth access token.

5. Required Pages & API Routes

The following pages and API routes are required. The API paths shown below must not be renamed or placed under a different API prefix because markers will test them directly.

Route	Access	Purpose
/	Public	Explains AI Capsule
/login	Public	Starts OAuth login
/dashboard	Protected	Shows the authenticated user's records
/api/health	Public	Returns { "status": "ok" }
GET /api/capsules	Protected	Read own records
POST /api/capsules	Protected	Create own record
PUT /api/capsules/:id	Protected	Update own record
DELETE /api/capsules/:id	Protected	Delete own record

Additional routes may be used as required for the OAuth login and callback process. However, the required /api/capsules CRUD routes must use the exact paths specified above and must be protected by JWT authentication middleware.

6. Prompt Data & Database

Each capsule belongs to the authenticated user. The user_id is the unique user identifier obtained from the OAuth provider (for example, the GitHub user ID). It must be obtained from the verified JWT for CRUD operations, not supplied by the browser.

Field	Description	Example
User Id	Authenticated user identifier obtained from the verified JWT; not entered by the user	GitHub user ID
Project name	Project / assignment	SmartFarm Irrigation
Prompt title	Short title	Debug cloud deployment
Prompt version	Version / iteration	v1, v2, v3
Prompt text	Actual prompt	Why does my Node server fail?
Response summary	AI response summary	Check start command
Category	Task type	Coding / Writing / Research
Usefulness	Student rating	Good / Needs Improvement
Reviewed	Response checked?	Yes / No

Field	Description	Example
Improved	Output improved?	Yes / No
Screenshot evidence	URL; upload optional	https://...
Notes	Reflection / comment	Tested and worked
Created at	Timestamp	Auto-generated

A simple text field such as v1, v2 or v3 is sufficient for the prompt version. Screenshot evidence may be provided as a URL; file upload is optional. You may adapt the database schema to suit your implementation, provided that all required information is retained and each record is clearly associated with the authenticated user.

```
CREATE TABLE capsules (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id TEXT NOT NULL,
  project_name TEXT NOT NULL,
  prompt_title TEXT NOT NULL,
  prompt_version TEXT,
  prompt_text TEXT NOT NULL,
  response_summary TEXT,
  category TEXT,
  usefulness TEXT,
  reviewed INTEGER DEFAULT 0,
  improved INTEGER DEFAULT 0,
  screenshot_url TEXT,
  notes TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

7. Required Application Behaviour

Your deployed application must support the complete CRUD cycle for an authenticated user.

- **CREATE:** add a new prompt record through the application and save it through POST /api/capsules.
- **READ:** load and display only the authenticated user's prompt records through GET /api/capsules.
- **UPDATE:** change an existing prompt record through PUT /api/capsules/:id. The user must not be able to update another user's record.
- **DELETE:** remove an existing prompt record through DELETE /api/capsules/:id. The user must not be able to delete another user's record.
- **Protected dashboard:** a user who has not successfully logged in must not be able to use the protected application or retrieve capsule data.
- **Health check:** GET /api/health remains public and returns { "status": "ok" } so the deployed backend can be checked independently.

Optional features such as search, filtering, charts, richer styling, CI/CD, refresh tokens, multiple tables and file upload may be added, but they do not replace any required behaviour and do not earn marks merely because they are more complex.

8. Cloud Deployment & Storage

After the application works locally, you need to deploy it to a real public Internet URL. The deployed version must include the React frontend, Express API, OAuth login, JWT-protected CRUD and /api/health.

For the simplest deployment, we recommend serving both the React frontend and the Express API from the same deployed application and public URL. This avoids unnecessary CORS and cross-origin cookie configuration.

You may use Render, Azure App Service or another genuine cloud/VPS/managed platform. Render and Azure are recommended because they provide straightforward paths for this assignment. Choosing a VPS,

container platform or other harder deployment does not earn extra marks; the required application behaviour is what is assessed.

Platform	Guidance
Render	Recommended. Render's free web service is suitable for this assignment. SQLite works normally, but its data may be lost after a restart or redeployment because the local filesystem is temporary; explain this limitation in your README. Render PostgreSQL may provide persistent storage, but free offers are time-limited and subject to change.
Azure App Service	Recommended/accepted. Eligible university students may be able to use Azure for Students credit without a credit card. Use free-tier services where possible to avoid unnecessary charges, and always check Microsoft's current offers, eligibility requirements and usage limits before creating resources.
Other cloud / VPS / managed platform	Accepted, provided that the application is genuinely deployed to a public URL and meets all assignment requirements. Choosing a more complex or difficult deployment method does not earn additional marks.

Free cloud services are recommended to avoid unnecessary costs. Check the provider's current free-tier limits and conditions before deployment, as these may change. You are not expected to pay for cloud hosting. Keep your deployed application available until the marking period is complete. A short delay while a free service starts up is acceptable.

9. OAuth, JWT & Backend Protection

GitHub OAuth is the recommended login method for this assignment. If you have problems using GitHub OAuth, you may use Google OAuth instead. No prior approval is required, but state your reason briefly in the README.

After a successful OAuth login, the Express backend must create an application JWT and store it in a Secure, HttpOnly cookie named token. All protected API routes must verify this JWT before allowing access to capsule data.

- GET, POST, PUT and DELETE routes under `/api/capsules` must all use JWT authentication middleware.
- Requests with no JWT or an invalid JWT must return 401 Unauthorized and must not return protected capsule data.
- The authenticated user must be identified from the verified JWT. Do not accept `user_id` from the frontend.
- CREATE must save the authenticated user as the record owner. READ, UPDATE and DELETE must only operate on records owned by that user.
- JWT_SECRET, OAuth client secrets and database credentials must be stored in environment variables and must not be committed to GitHub.
- The application JWT must be stored in a Secure, HttpOnly cookie named token. Do not use localStorage or an Authorization Bearer token instead.

You do not need to extract, display or submit your real JWT. Successful CRUD after OAuth login demonstrates that a valid authenticated session can use the protected API.

Use the following two cURL checks against the deployed GET `/api/capsules` endpoint before submission:

```
# Test 1 - no authentication
curl -i https://YOUR-APP/api/capsules
# Required: 401 Unauthorized

# Test 2 - fake / invalid JWT
curl -i -H "Cookie: token=fake-token-123" https://YOUR-APP/api/capsules
# Required: 401 Unauthorized
```

Test 1 checks that the backend requires authentication. Test 2 checks that the backend actually validates the JWT rather than only checking whether a cookie exists. Only GET needs to be demonstrated with cURL in the video. Markers/tutors will also inspect the submitted source code to confirm that POST, PUT and DELETE use the same protection, and may repeat the public cURL checks against your deployed URL.

10. Submission

1. Source-Code ZIP

- Submit your complete project as one .zip file, including all files required for assessment such as package.json and database setup files.
- Exclude node_modules, build folders and other generated dependencies.
- ZIP only. RAR, 7Z, PDF, DOCX or external links are not substitutes for the required source-code ZIP.

2. Screen-Recorded Video

- Submit a 3-5 minute .mp4 video with working audio (e.g. Zoom or Teams meeting recording).
- MP4 only. External video links or other video formats are not substitutes for the required MP4 upload.
- Check the uploaded video before submission and confirm that both video and audio work.

You must submit both the source-code ZIP file and the MP4 video directly to the LMS so that your submission can be assessed independently.

11. README Requirements

Your README is part of the assessment evidence. It must give the marker enough information to install the project, understand the implementation and access the deployed application.

- The public deployed URL and the cloud platform used.
- Installation and run instructions, including the required npm install / npm run commands and any frontend/backend start commands.
- The mandatory API routes and how the React frontend communicates with Express.
- The OAuth provider used and how the Express application JWT is issued, stored and verified.
- The environment-variable names used, without secret values.
- How the database is created/initialised, how user ownership is stored and whether the deployed storage is persistent or ephemeral.
- The two required cURL commands and the results you obtained.
- One honest limitation of the submitted application.

Never place real passwords, OAuth client secrets, JWT_SECRET values, database passwords or a real JWT value in the README or repository.

12. AI-Assisted Development

AI tools may be used extensively for project setup, React components, Express routes, SQLite queries, OAuth/JWT integration, cloud deployment, CSS, testing and debugging.

Your README must include:

- the AI tool or tools used;
- at least one problem found and corrected in AI-generated code or configuration;
- how OAuth login, JWT verification and protected API behaviour were verified;

- how CRUD behaviour and user data ownership were verified;
- one implementation or deployment decision you made and can explain independently.

You remain responsible for the submitted work. AI-generated code or configuration must still satisfy all stated security, CRUD and deployment requirements, and you must be able to explain the implementation you submit.

13. Video Demonstration

The video is the main evidence that your application was successfully deployed and working on the public Internet. Demonstrate the deployed application, not a localhost copy.

- Open the deployed public URL and show that it is not localhost. Show `/api/health` returning `{ "status": "ok" }`.
- Before logging in, run cURL Test 1 against `GET /api/capsules` and show 401 Unauthorized.
- Run cURL Test 2 using `token=fake-token-123` and show 401 Unauthorized.
- Complete GitHub OAuth login, or the accepted Google fallback, and enter the protected dashboard.
- Demonstrate complete CRUD in the deployed application: CREATE, READ, UPDATE and DELETE a prompt record.
- Show the cloud platform settings and relevant environment-variable names, but do not show any secret values.
- Briefly explain your database/storage approach, any persistence or ephemeral-storage limitation, and one deployment issue or limitation.

Do not display or submit your real JWT. Successful CRUD after OAuth login is sufficient evidence that an authenticated session can access the protected API.

14. Marking - 100 Marks

A localhost-only submission can receive marks for Section A only. Sections B–E require evidence of a working cloud deployment. If you do not provide a working public URL and video evidence of the deployed application, the maximum mark is 35/100.

A — Full-stack CRUD (35)	
Criterion	Marks
React frontend communicates with Express API	5
Create record (POST)	7
Read own records (GET)	7
Update record (PUT)	7
Delete record (DELETE)	7
Appropriate database integration / record model	2
Section Total	35
B — Real cloud deployment (25)	
Criterion	Marks
Public HTTPS URL works and is not localhost	10
Frontend and Express backend/API work in cloud	7
Public <code>/api/health</code> returns required response	3
Cloud runtime/settings/deployment evidence shown	5

Section Total	25
C — OAuth + JWT authentication (15)	
Criterion	Marks
GitHub OAuth works (or Google fallback)	5
Express issues its own application JWT after OAuth	4
JWT is verified server-side for protected routes	3
HttpOnly/Secure cookie and secrets handled correctly	3
Section Total	15
D — Protected API & security evidence (15)	
Criterion	Marks
GET /api/capsules without authentication returns 401	4
GET /api/capsules with fake/invalid JWT returns 401	4
All CRUD routes protected by JWT middleware (code inspection)	4
Ownership uses verified JWT identity; users restricted to own records	3
Section Total	15
E — README, video & cloud configuration (10)	
Criterion	Marks
README covers setup/run, deployment, API, OAuth/JWT, environment variables, storage, cURL tests and one application limitation.	3
Required video evidence is complete	3
Cloud configuration and environment-variable names are shown without revealing secrets.	2
AI-assisted development statement, one problem identified and corrected, and one implementation/deployment decision explained.	2
Section Total	10
Total Marks	100

15. Reference Materials

The following materials are provided as implementation references. They do not replace the requirements in this assignment specification.

- **JWT with Express** — Use the jsonwebtoken package documentation for signing and verifying JWTs: <https://www.npmjs.com/package/jsonwebtoken>
- **GitHub OAuth — Module 5 Lab** — Refer to the *Authentication, Identity and Secure Access for Web Applications* lab and its example code: <https://github.com/CSE3CWA-5006/CSE3CWA-5006-Week-05>
- **GitHub OAuth Web Application Flow** — Refer to the official GitHub documentation: <https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps>
- **Render Deployment — Module 2, Lab 1** — Refer to the lab materials and example code: <https://github.com/CSE3CWA-5006/CSE3CWA-5006-Week-01>
- **Deploying Node/Express on Render** — Refer to the official Render deployment guide: <https://render.com/docs/deploy-node-express-app>

Self-check: before you submit, watch your own video as if you were the marker. Can you clearly see the public deployed URL (not localhost), the /api/health response, both required cURL tests returning 401 Unauthorized, a successful OAuth login, complete CREATE, READ, UPDATE and DELETE behaviour, and the cloud configuration without revealing any secret values? If any required evidence is missing or unclear, re-record the video before submitting.

Common things markers will check: the application is genuinely deployed to a public URL; the frontend and Express backend are working online; /api/capsules rejects requests with no JWT or an invalid JWT; all CRUD routes are protected by JWT authentication middleware; users can access only their own records; OAuth login and authenticated CRUD work correctly; secrets are stored in environment variables and are not committed to the submitted source code; and the required cloud, cURL and application behaviour is clearly demonstrated in the video.

The guiding principle. AI Capsule is deliberately small. The goal is to build a complete full-stack CRUD application and prove that you can deploy, authenticate, protect, configure and explain it on a real public cloud platform. Focus on the required behaviour rather than unnecessary complexity.